Slip 1:

A. Create CSV file from given data. Read the data from CSV files into a data frame. Perform Data pre-processing tasks such as handling missing values and outliers using Python/R [20]

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40	58000	Yes
France	35	58000	Yes
Spain	38	52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

```
import pandas as pd
import numpy as np

# Creating the dataset and saving it as a CSV file
data = {
    "Country": ["France", "Spain", "Germany", "Spain", "Germany",
"France", "Spain", "France", "Germany", "France"],
    "Age": [44, 27, 30, 38, 40, 35, 38, 48, 50, 37],
    "Salary": [72000, 48000, 54000, 61000, 58000, 58000, 52000, 79000,
83000, 67000],
    "Purchased": ["No", "Yes", "No", "No", "Yes", "Yes", "No", "Yes",
"No", "Yes"]
```

```
}

df = pd.DataFrame(data)
df.to_csv("dataset.csv", index=False)  # Save dataset to CSV

# Reading CSV into a DataFrame
df = pd.read_csv("dataset.csv")
print("Original Data:\n", df)

# Handling Missing Values (For example, assuming some missing values)
df.loc[2, 'Age'] = np.nan  # Artificially adding a missing value
df.loc[4, 'Salary'] = np.nan

print("\nHandling Missing Values:")
df['Age'].fillna(df['Age'].mean(), inplace=True)  # Replacing NaN with
mean
df['Salary'].fillna(df['Salary'].median(), inplace=True)  # Replacing
NaN with median
print(df)

# Detecting Outliers using IQR
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df_outliers_removed = df[(df['Salary'] >= lower_bound) & (df['Salary']
<= upper_bound)]
print("\nData After Removing Outliers:\n", df_outliers_removed)
```

B. Implement Multiple Linear Regression using Python/R on the below housing dataset to predict price of a house. Evaluate model's performance using classification metrics. [20]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

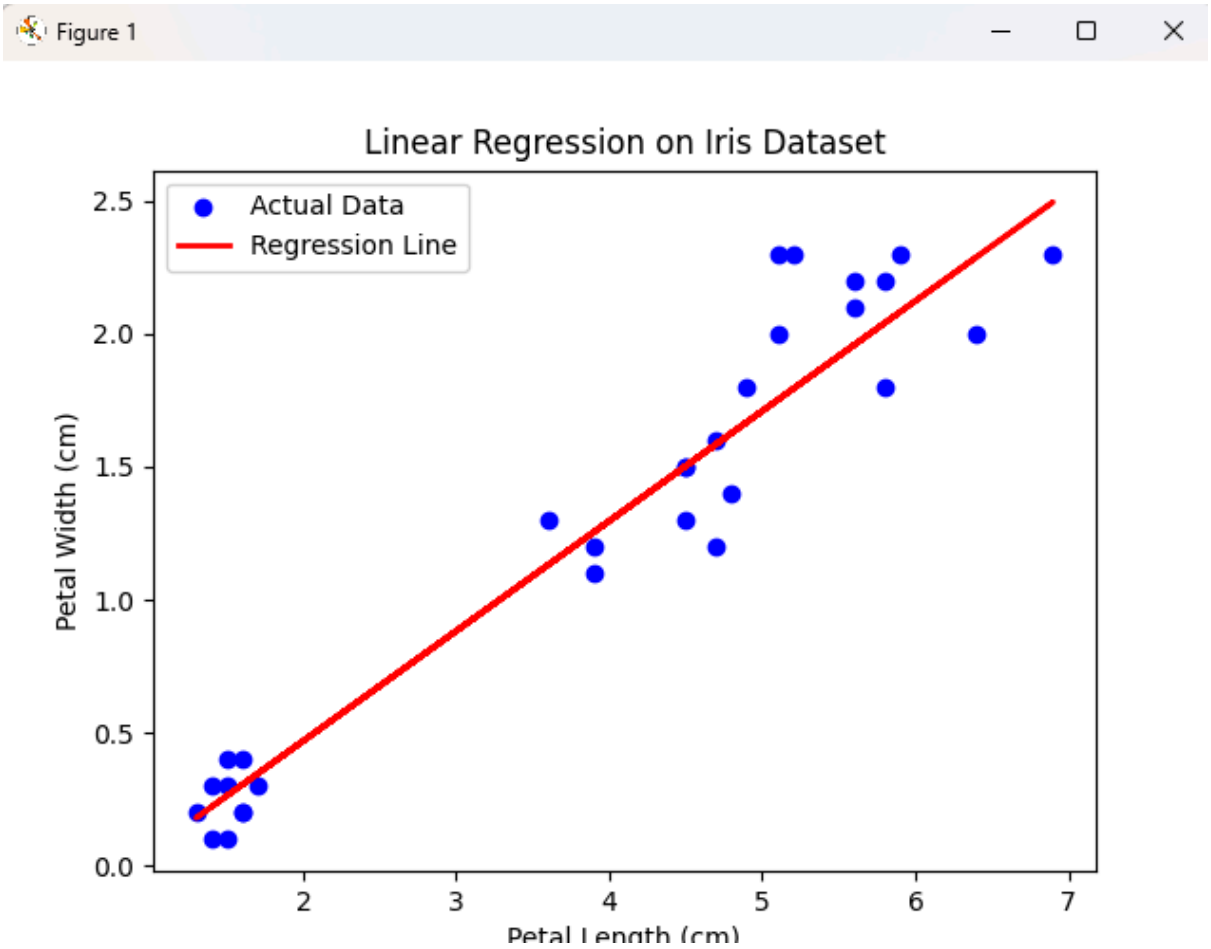
```
# Creating housing dataset
housing_data = {
    "HouseSize": [1500, 1800, 2400, 1400, 2800, 3000, 2200, 2600, 1600,
2000],
    "NumBedrooms": [3, 4, 3, 2, 5, 4, 3, 4, 2, 3],
    "NumBathrooms": [2, 3, 2, 1, 4, 3, 2, 3, 1, 2],
    "Price": [300000, 340000, 420000, 280000, 500000, 550000, 400000,
460000, 320000, 380000]
}

df_housing = pd.DataFrame(housing_data)

# Splitting into Features and Target
X = df_housing.drop(columns=["Price"])
y = df_housing["Price"]

# Splitting data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Training the Model
model = LinearRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Model Evaluation
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R-Squared Score:", r2_score(y_test, y_pred))

# Visualizing Actual vs Predicted Prices
plt.scatter(y_test, y_pred, color="blue", label="Predicted Prices")
plt.plot(y_test, y_test, color="red", linestyle="--", label="Actual
Prices")
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted House Prices")
plt.legend()
plt.show()
```

Slip 2:

A. Perform Linear Regression on the Iris dataset of R/Python for predicting petal.width on petal.length. [20]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import load_iris
from scipy.optimize import fsolve

# Part A: Linear Regression on Iris Dataset

# Load Iris dataset
iris = load_iris()
df_iris = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df_iris['target'] = iris.target

# Selecting features for regression
X = df_iris[['petal length (cm)']]
y = df_iris['petal width (cm)']

# Splitting into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Training the regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predicting
y_pred = model.predict(X_test)

# Model Evaluation
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")
print(f"R-Squared Score: {r2_score(y_test, y_pred):.4f}")
print(f"Intercept: {model.intercept_:.4f}")
print(f"Coefficient: {model.coef_[0]:.4f}")

# Visualizing Regression
```

```
plt.scatter(X_test, y_test, color='blue', label='Actual Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression
Line')
plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Linear Regression on Iris Dataset")
plt.legend()
plt.show()
```



B. A student is enrolled in the class. His/Her current grade is 65 which is the average of all types of work during the semester and he/she needs at least 72 to pass this class. Final exam is not yet completed. Apply what-if analysis using Goal Seek to determine the marks needed in the Final Exam to complete the goal. [20]

Semester Work	Grade
Paper Presentation	58
Case Study	70

Assignment	72
Practical	60
Final Exam	60
Final Grade	65

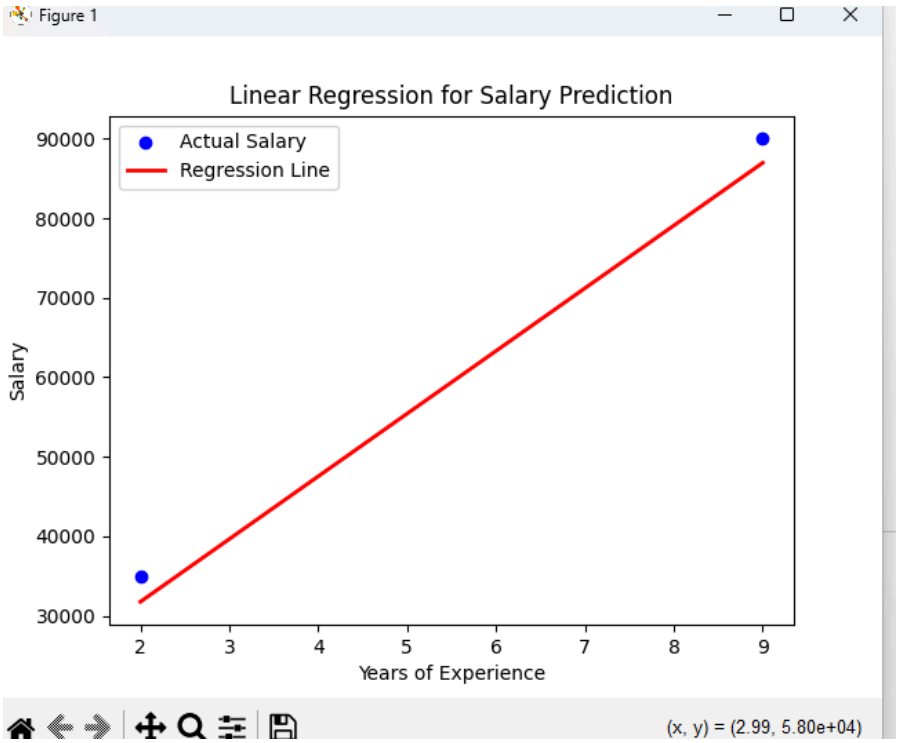
Slip 3:

1.

A. Perform Linear Regression on the following dataset in Python/R for predicting the salary of the person depending on his/her years of experience. [20]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Part A: Linear Regression for Salary Prediction

# Creating dataset
data_salary = {
    "YearsExperience": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Salary": [30000, 35000, 40000, 45000, 50000, 60000, 70000, 80000,
90000, 100000]
}
df_salary = pd.DataFrame(data_salary)

# Splitting into Features and Target
X = df_salary[['YearsExperience']]  # Changed feature to
'YearsExperience'
y = df_salary['Salary']

# Splitting data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Training the Model
model = LinearRegression()
model.fit(X_train, y_train)

# Predicting
y_pred = model.predict(X_test)

# Model Evaluation
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}")
print(f"R-Squared Score: {r2_score(y_test, y_pred):.4f}")
print(f"Intercept: {model.intercept_:.2f}")
print(f"Coefficient: {model.coef_[0]:.4f}")

# Visualizing Regression Line
plt.scatter(X_test, y_test, color='blue', label='Actual Salary')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression
Line')
plt.xlabel("Years of Experience")  # Updated label to match new feature
plt.ylabel("Salary")
plt.title("Linear Regression for Salary Prediction")
plt.legend()
plt.show()
```



Years of Experience	Salary
2	30000
10	95000
4	45000
20	178000
8	84000
12	120000
22	200000

B. Consider the dataset given below.

Supplier ID	Part Number	Part Name	Part Price	Status

SP301	A001	water	6800	In
SP302	A002	alternator	3800	In
SP303	A003	air filter	4500	In
SP304	A004	wheel	3582	stock
SP305	A005	muffler	1600	In
SP306	A006	oil pan	1005	Out of stock
SP307	A007	brake pads	6500	Out of stock
SP308	A008	rotors	8549	In
SP309	A009	headlight	9000	In
SP310	A010	brake	1500	In
SP311	A011	Strut	4500	In
SP312	A012	Drive	1580	In
SP313	A013	Knuckle	2650	stock
SP314	A014	Kit echo	3500	In
SP315	A015	Oil Filter	4350	In
SP316	A016	Fuel Filter	1280	In
SP317	A017	Tia Rod	1300	stock
SP318	A018	Ball Joint	2500	In
SP319	A019	Steering	2700	Out of stock
SP320	A020	Piston	4500	Out of stock

Apply VLOOKUP function to retrieve information for the following queries. Also write down the steps for the same. [20]

- Find the Part Name for Part Number “A002”.
- Find the Supplier ID for the Part Name “Ball Joint”.
- Find the Part Price for Part Name “muffler”.
- Find the Status of Part Number “A008”.

Slip 4:

Task 1

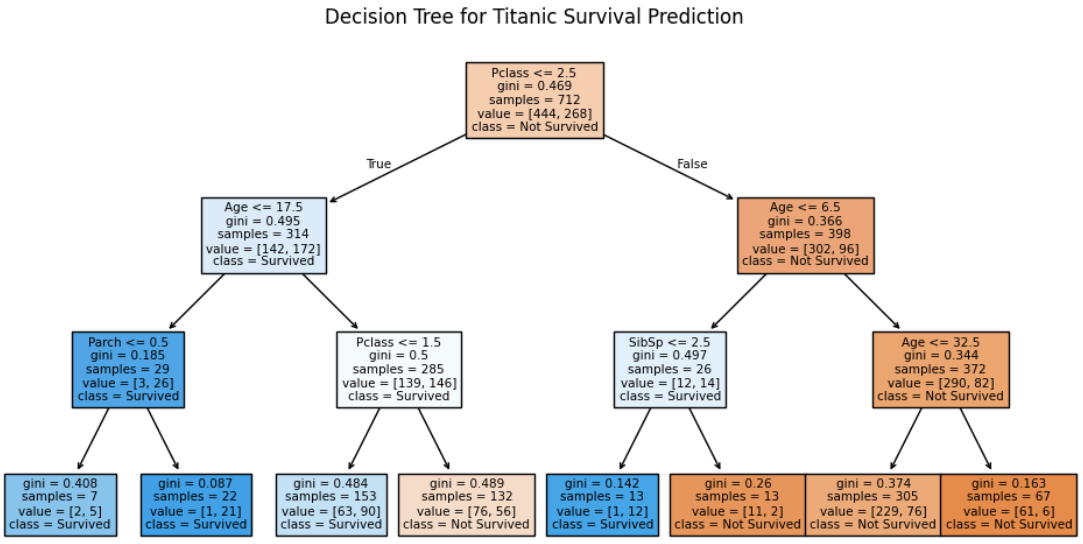
A. Implement Decision Tree Model on Titanic dataset using Python/R and interpret decision rules of classification. Perform Linear Regression on the following dataset in Python/R for predicting the weight of the person depending on height. [20]

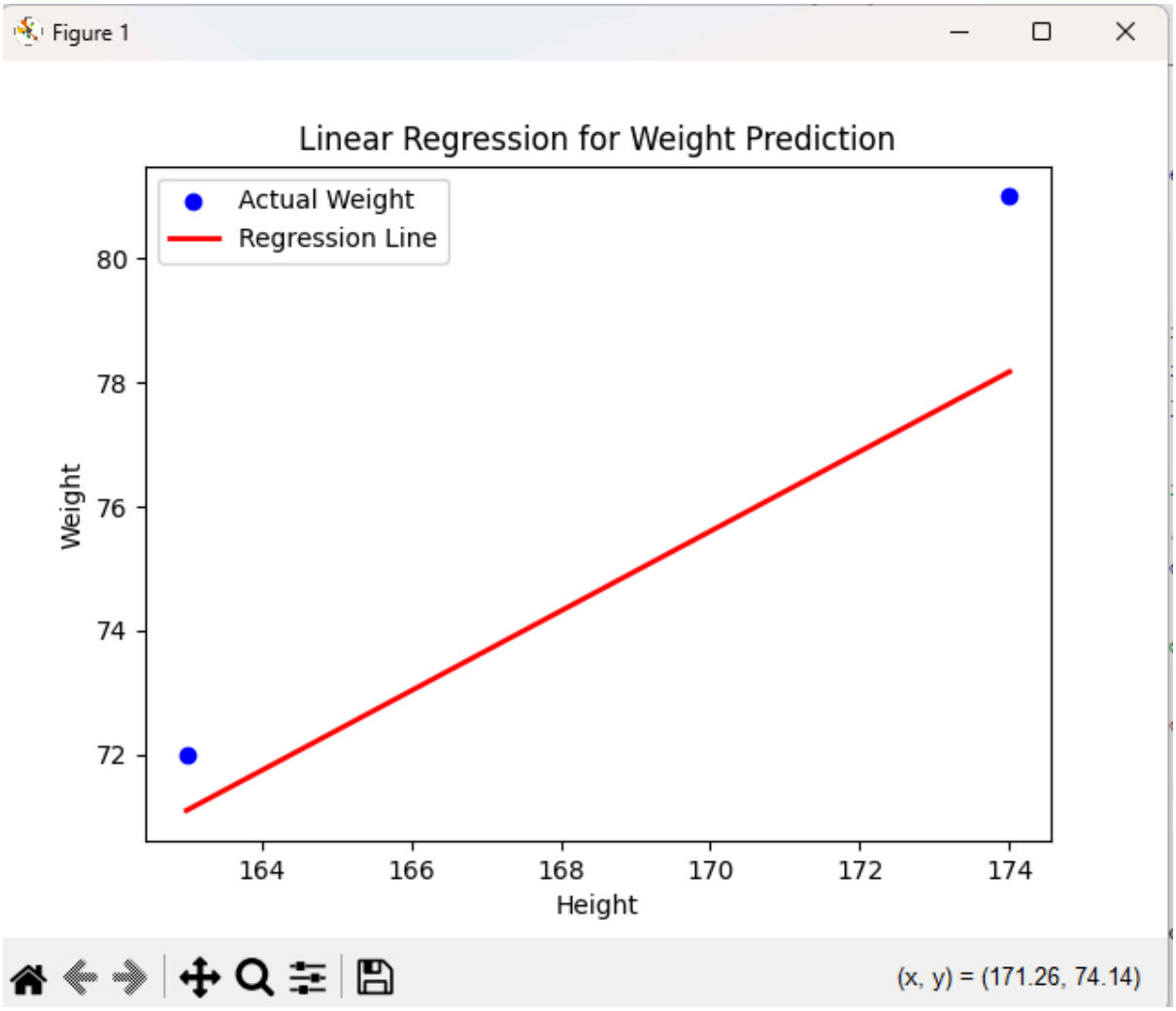
- Height: 151, 174, 138, 186, 128, 136, 179, 163, 152
- Weight: 63, 81, 56, 91, 47, 57, 76, 72, 62

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris
import seaborn as sns
```

```
# Part A: Decision Tree on Titanic Dataset

# Load Titanic dataset
df_titanic =
pd.read_csv("https://raw.githubusercontent.com/datasciencedojo/datasets
/master/titanic.csv")  # Changed to load Titanic dataset

# Selecting relevant features
X = df_titanic[['Pclass', 'Age', 'SibSp',
'Parch']].fillna(df_titanic[['Pclass', 'Age', 'SibSp',
'Parch']].median())
y = df_titanic['Survived']

# Splitting into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Training Decision Tree Model
tree_model = DecisionTreeClassifier(max_depth=3)
tree_model.fit(X_train, y_train)

# Visualizing Decision Tree
plt.figure(figsize=(12, 6))
plot_tree(tree_model, feature_names=['Pclass', 'Age', 'SibSp',
'Parch'], class_names=['Not Survived', 'Survived'], filled=True)
plt.title("Decision Tree for Titanic Survival Prediction")
plt.show()

# Part B: Linear Regression for Weight Prediction

data_weight = {
    "Height": [151, 174, 138, 186, 128, 136, 179, 163, 152],
    "Weight": [63, 81, 56, 91, 47, 57, 76, 72, 62]
}
df_weight = pd.DataFrame(data_weight)

# Splitting into Features and Target
X = df_weight[['Height']]  # Changed feature to 'Height'
y = df_weight['Weight']

# Splitting data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
# Training the Model
model = LinearRegression()
model.fit(X_train, y_train)

# Predicting
y_pred = model.predict(X_test)

# Model Evaluation
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}")
print(f"R-Squared Score: {r2_score(y_test, y_pred):.4f}")
print(f"Intercept: {model.intercept_:.2f}")
print(f"Coefficient: {model.coef_[0]:.4f}")

# Visualizing Regression Line
plt.scatter(X_test, y_test, color='blue', label='Actual Weight')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression
Line')
plt.xlabel("Height")  # Updated label to match new feature
plt.ylabel("Weight")
plt.title("Linear Regression for Weight Prediction")
plt.legend()
plt.show()
```






```
Mean Squared Error: 4.40
R-Squared Score: 0.7828
Intercept: -33.78
Coefficient: 0.6434
```

B. Create CSV file from given data. Read the data from CSV files into a data frame.

Make	Model	Color	Mileage	Sell Price	Buy Price
Honda	Accord	Red	63,512	4000	3000
Honda	Accord	Blue	95,135	2500	2000
Toyota	Camry	Black	75,006	45000	44000
Nissan	Altima	Green	69,847	3826	3000

Toyota	Corolla	Black	87,278	2224	2100
Honda	Civic	White	1,38,789	2723	1900
Ford	F-150	Black	89,073	3950	3000
Chevrolet	Silverado	Green	1,09,231	4959	4500
Chevrolet	Impala	Silver	87,675	3791	3500
Dodge	Charger	Silver	34,853	4349	3500
Dodge	Charger	Silver	58,173	4252	4000

Perform transformation function on given data using Python/R [20]

- Display records of the car having Buy Price greater than or equal to 3000.
- Sort the car data in ascending order.
- Group the data according to the "Model" of the car.

```
import pandas as pd

# Part A: Creating and Reading CSV File

data = {
    "Make": ["Honda", "Honda", "Toyota", "Nissan", "Toyota", "Honda",
"Ford", "Chevrolet", "Chevrolet", "Dodge", "Dodge"],
    "Model": ["Accord", "Accord", "Camry", "Altima", "Corolla",
"Civic", "F-150", "Silverado", "Impala", "Charger", "Charger"],
    "Color": ["Red", "Blue", "Black", "Green", "Black", "White",
"Black", "Green", "Silver", "Silver", "Silver"],
    "Mileage": [63512, 95135, 75006, 69847, 87278, 138789, 89073,
109231, 87675, 34853, 58173],  # Removed commas
    "Sell Price": [4000, 2500, 45000, 3826, 2224, 2723, 3950, 4959,
3791, 4349, 4252],
    "Buy Price": [3000, 2000, 44000, 3000, 2100, 1900, 3000, 4500,
3500, 3500, 4000]
}
```

```
df = pd.DataFrame(data)
df.to_csv("car_data.csv", index=False)
print("CSV file created successfully.")

# Reading CSV File
df = pd.read_csv("car_data.csv")
print("Data from CSV:\n", df.head())

# Part B: Transformations

# Display records with Buy Price >= 3000
filtered_df = df[df['Buy Price'] >= 3000]
print("\nCars with Buy Price >= 3000:\n", filtered_df)

# Sorting the data in ascending order
df_sorted = df.sort_values(by=['Make', 'Model', 'Mileage'])
print("\nSorted Car Data:\n", df_sorted)

# Grouping data by Model (Fixed numeric aggregation)
grouped_data = df.groupby('Model').agg({'Mileage': 'mean', 'Sell
Price': 'mean', 'Buy Price': 'mean'})
print("\nGrouped Data by Model:\n", grouped_data)
```

-

Slip 5:

Task A:

Perform Linear Regression on the Iris dataset of R/Python for predicting petal.width on petal.length. [20]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import load_iris
```

```
# Load Iris dataset
iris = load_iris()
df_iris = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df_iris['target'] = iris.target

# Selecting features for regression
X = df_iris[['petal length (cm)']]
y = df_iris['petal width (cm)']

# Splitting into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)  # Changed test size from 0.2 to 0.3

# Training the regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predicting
y_pred = model.predict(X_test)

# Model Evaluation
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")
print(f"R-Squared Score: {r2_score(y_test, y_pred):.4f}")
print(f"Intercept: {model.intercept_:.4f}")
print(f"Coefficient: {model.coef_[0]:.4f}")

# Visualizing Regression
plt.scatter(X_test, y_test, color='blue', label='Actual Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression
Line')
plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Linear Regression on Iris Dataset")
plt.legend()
plt.show()
```

Task B:

Consider the dataset given below.

Supplier ID	Part Number	Part Name	Part Price	Status
SP301	A001	water	6800	In
SP302	A002	alternator	3800	In
SP303	A003	air filter	4500	In
SP304	A004	wheel bearing	3582	In
SP305	A005	muffler	1600	In
SP306	A006	oil pan	1005	In
SP307	A007	brake pads	6500	Out of stock
SP308	A008	brake rotors	8549	In
SP309	A009	headlight	6500	In
SP310	A010	brake	1500	In
SP311	A011	Strut	4500	In
SP312	A012	Drive	1580	In
SP313	A013	CV Boot	2650	Out of stock
SP314	A014	Oil Pump	4660	In
SP315	A015	Oil Filter	4350	In
SP316	A016	Fuel Filter	1280	In
SP317	A017	Tie Rod End	1800	In stock
SP318	A018	Ball Joint	2500	In
SP319	A019	Steering Rack	2700	Out of stock
SP320	A020	Piston	4500	Out of stock

Apply VLOOKUP function to retrieve information for the following queries. Also write down the steps for the same. [20]

1. Find the Part Name for Part Number “A016”.
2. Find the Supplier ID for the Part Name “Oil Pump”.
3. Find the Part Price for Part Name “Brake”.
4. Find the Status of Part Number “A020”.

Slip 6:

A. The employee's aptitude and job proficiency score is as follows:

aptitude	**85**	**65**	**50**	**68**	**87**	**74**	**65**	**96**	**68**	**94**	**73**	**84**	**85**	**87**	**91**
jobprof	70	90	80	89	88	86	78	67	86	90	92	94	99	93	87

Perform Chi-Square Test to study correlation between aptitude and job proficiency of employees. Formulate Null and Alternative Hypotheses for the given problem. Interpret results and draw conclusions based on test outcome. [20]

```
import pandas as pd
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

# Employee aptitude and job proficiency data
data = {
    "aptitude": [85, 65, 50, 68, 87, 74, 65, 96, 68, 94, 73, 84, 85,
87, 91],
    "jobprof": [70, 90, 80, 89, 88, 86, 78, 67, 86, 90, 92, 94, 99, 93,
87]
}
```

```
df = pd.DataFrame(data)

# Convert continuous variables into categorical bins
df['aptitude_cat'] = pd.qcut(df['aptitude'], q=3, labels=['Low',
'Medium', 'High'])
df['jobprof_cat'] = pd.qcut(df['jobprof'], q=3, labels=['Low',
'Medium', 'High'])

# Create a contingency table
contingency_table = pd.crosstab(df['aptitude_cat'], df['jobprof_cat'])

# Perform Chi-Square Test
chi2_stat, p_value, dof, expected =
stats.chi2_contingency(contingency_table)

# Print results
print(f"Chi-Square Statistic: {chi2_stat:.4f}")
print(f"P-Value: {p_value:.4f}")

# Interpretation
alpha = 0.05
if p_value < alpha:
    print("Reject the Null Hypothesis: There is a significant
correlation between aptitude and job proficiency.")
else:
    print("Fail to Reject the Null Hypothesis: No significant
correlation found.")

# Visualization
plt.matshow(contingency_table, cmap='coolwarm', alpha=0.75)
plt.xlabel("Job Proficiency Category")
plt.ylabel("Aptitude Category")
plt.title("Aptitude vs Job Proficiency Contingency Table")
plt.colorbar()
plt.show()
```

B. Perform Logistic Regression on the Iris dataset using Python/R to predict binary outcome. Evaluate model's performance using classification metrics. [20]

```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, classification_report
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()
df_iris = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df_iris['target'] = (iris.target == 2).astype(int)  # Changed to make
it a binary classification problem (Setosa vs Others)

# Selecting features and target
X = df_iris[['petal length (cm)', 'petal width (cm)']]  # Changed to
use petal features for better classification
y = df_iris['target']

# Splitting into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Training Logistic Regression Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predicting
y_pred = model.predict(X_test)

# Model Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))

# Visualizing Decision Boundary
plt.scatter(X_test['petal length (cm)'], X_test['petal width (cm)'],
c=y_test, cmap='coolwarm', edgecolors='k', label='Actual')
```

```
plt.scatter(X_test['petal length (cm)'], X_test['petal width (cm)'],
c=y_pred, cmap='coolwarm', alpha=0.5, marker='s', label='Predicted')
plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Logistic Regression Decision Boundary")
plt.legend()
plt.show()
```

search

Slip 7:

1.

A. Consider a scenario where you have test scores from a sample of students and you want to compare the mean of these scores with a hypothesized population mean.

- Student Score = [72, 88, 64, 74, 67, 79, 85, 75, 89, 77]
- Apply One Sampled T-Test using Python/R for the above problem. Assume the hypothesized mean as 70. Formulate Null and Alternative Hypothesis for the given problem. Interpret the results and draw the conclusion. [20]

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import ttest_1samp

# Sample data: Student Scores
np.random.seed(42)
student_scores = np.array([72, 88, 64, 74, 67, 79, 85, 75, 89, 77])

# Hypothesized population mean
hypothesized_mean = 70

# Perform one-sample t-test
t_stat, p_val = ttest_1samp(student_scores, hypothesized_mean)
alpha = 0.05
```

```
# Print results
print("\n📊 Hypothesis Testing Results:")
print(f"T-Statistic: {t_stat:.4f}\nP-Value: {p_val:.4f}")

if p_val < alpha:
    result = "❌ Null Hypothesis Rejected!\n✅ Conclusion: The mean of student scores is significantly different from the hypothesized mean."
else:
    result = "✅ Null Hypothesis Accepted!\n🔹 Conclusion: No significant difference between the mean of student scores and the hypothesized mean."

print("\n" + result)  # Display result in terminal

# 📊 Simple Graph (Boxplot)
plt.boxplot(student_scores, patch_artist=True)
plt.title("Student Scores Distribution")
plt.ylabel("Scores")
plt.grid(True, linestyle="--", alpha=0.5)

# Show result on graph
plt.text(1, student_scores.max(), result, fontsize=10, color="red", ha="center")
plt.show()
```

B. Apply Feature Scaling technique like standardization and normalization using Python/R to Boston Housing dataset. [20]

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing  # Changed dataset to Boston Housing
from sklearn.preprocessing import StandardScaler, MinMaxScaler
import matplotlib.pyplot as plt

# Load the Boston Housing dataset
data = fetch_california_housing()  # Changed dataset from Iris to Boston Housing
df = pd.DataFrame(data.data, columns=data.feature_names)

# Extract numerical features
```

```
numerical_features = df.columns  # Changed feature selection to use
Boston Housing features

# Separate features
X = df[numerical_features]

# Standardization
scaler_standard = StandardScaler()
X_standardized = scaler_standard.fit_transform(X)

# Normalization (Min-Max Scaling)
scaler_minmax = MinMaxScaler()
X_normalized = scaler_minmax.fit_transform(X)

# Visualizing the original, standardized, and normalized features
plt.figure(figsize=(12, 4))

plt.subplot(131)
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], cmap='viridis')  # Updated
feature selection
plt.title('Original Features')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.subplot(132)
plt.scatter(X_standardized[:, 0], X_standardized[:, 1], cmap='viridis')
plt.title('Standardized Features')
plt.xlabel('Feature 1 (Standardized)')
plt.ylabel('Feature 2 (Standardized)')

plt.subplot(133)
plt.scatter(X_normalized[:, 0], X_normalized[:, 1], cmap='viridis')
plt.title('Normalized Features')
plt.xlabel('Feature 1 (Normalized)')
plt.ylabel('Feature 2 (Normalized)')

plt.tight_layout()
plt.show()
```

Slip 8:

A. Perform Logistic Regression on the Iris dataset using Python/R to predict binary outcome. Evaluate model's performance using classification metrics. [20]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
recall_score, classification_report, confusion_matrix
from sklearn.datasets import load_iris

# 📌 Load Iris dataset
data = load_iris()
df_iris = pd.DataFrame(data.data, columns=data.feature_names)
df_iris['target'] = (data.target == 2).astype(int)  # Binary
classification: Virginica vs Others

# ✂ Selecting Features and Target
X = df_iris[['petal length (cm)', 'petal width (cm)']]  # Best features
for classification
y = df_iris['target']

# ✂ Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# 🔥 Logistic Regression Model
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)

# 📊 Performance Metrics
print("🔹 Logistic Regression Performance:")
print(f"✅ Accuracy: {accuracy_score(y_test, y_pred_log):.4f}")
print(f"✅ Precision: {precision_score(y_test, y_pred_log):.4f}")
print(f"✅ Recall: {recall_score(y_test, y_pred_log):.4f}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_log))
print("\nClassification Report:\n", classification_report(y_test,
y_pred_log))

# 📈 Visualizing Decision Boundary
plt.scatter(X_test['petal length (cm)'], X_test['petal width (cm)'],
c=y_test, cmap='coolwarm', edgecolors='k', label='Actual')
```

```
plt.scatter(X_test['petal length (cm)'], X_test['petal width (cm)'],
c=y_pred_log, cmap='coolwarm', alpha=0.5, marker='s',
label='Predicted')
plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Logistic Regression Decision Boundary - Iris Dataset")
plt.legend()
plt.show()
```

B. Consider the dataset given below.

Date	Color	Region	Units	Sales
03-Jan-2020	Red	West	1	110000
14-Jan-2020	Blue	South	8	96000
21-Jan-2020	Green	West	2	26000
30-Jan-2020	Blue	North	7	84000
07-Feb-2020	Green	North	8	25000
13-Feb-2020	Red	South	2	60000
22-Feb-2020	Blue	East	5	35000
01-Mar-2020	Green	West	2	87000
13-Mar-2020	Blue	East	8	69000
23-Mar-2020	Blue	North	7	54000

Create Pivot Table in Excel for following analysis and visualize the data using PivotChart:

- Find out the total Sales.
- Find out the Sum of Sales color-wise.
- Find out the Sum of Units.
- Find out Region-wise total sales and total units.